The Total-Work Cost of Bounded Retry Guarantees

ANONYMOUS AUTHOR(S)

Bounding retries can move computation under protection; reducing protected work can increase total work. We characterize this tradeoff for persistent whole-kernel jobs with specified completion and observation operations. With a supplied write bound, independent jobs admit a complete polynomial total/protected-work frontier for every retry allowance; sufficient retries extend it to all dependency DAGs. One budget-unaware policy attains the least total-work curve under optimal protection at every budget for independent jobs. A full-program domination proof supports finite Pareto synthesis elsewhere. We also count protected entry and comparison costs in both resources: one-write policies have a complete order/subset representation, but deciding joint caps becomes NP-complete in this added-cost class. Java wrappers realize the core contract. Linux pathname traversal supplies a conditional source-work upper bound and exposes sensitivity to the call-counting convention. Roslyn preservation checks have mixed timings. The results identify exact resource choices under explicit interface assumptions, without a latency guarantee.

CCS Concepts: • **Software and its engineering → Concurrent programming structures**.

Additional Key Words and Phrases: concurrent programming, retries, resource guarantees, scheduling, online policies

1 Introduction

An optimistic loop prepares outside protection, validates its snapshot and publishes on a match. Recomputing under protection bounds retries but may protect all remaining work. Cached completion instead reuses preparation on a match and recomputes on a mismatch, paying for both executions. What is the least total work needed for a given protection guarantee?

We study persistent jobs with fixed-work opaque kernels, immutable own inputs and saved predecessor outputs. Cheap conditional publication, fresh protected completion and cached completion consume calls Q, total kernel work W and protected kernel work L. Completed jobs persist; waiting is excluded.

Take works 8 and 16 with two calls. At supplied $B = 1$, order $8 \to 16$ attains worst $(W, L) = (32, 16)$; the reverse requires $(40, 16)$ at the same protection cap. Independent jobs attain the first pair. Simultaneously optimal protection at every hidden budget instead requires $(40, 16)$ in both orders. All-fresh gives $(24, 24)$. These distinct contracts need not have coincident coordinate maxima.

We prove complete polynomial frontiers for independent jobs and enough-retry DAGs, and one unaware independent-job policy with the least work curve under simultaneous optimal protection. Precedence and observations define boundaries. A domination theorem makes finite immediate-preparation trees complete elsewhere, retaining free inspections, old snapshots and paid retained records. Including entry/comparison costs in both resources gives a complete one-write representation but makes joint-cap decision NP-complete.

Fallback and synchronization choice are established [6, 41, 42], as are testing/execution tradeoffs [35, 38] and Pareto minimax synthesis [12, 15]. Section 10 gives the precise testing correspondence. Our contribution is the completion-program reduction and exact resource contracts.

2 Completion Interfaces and Resource Contracts

Write $\mathbb{N} = \{0, 1, \ldots\}$. Jobs $J = \{1, \ldots, n\}$, $n \geq 1$, form a finite DAG with predecessor set $\mathrm{Pred}(i)$. Readiness requires all predecessors completed. Completion records an exact immutable output and persists despite later external writes. The objective is one successful foreground transformation per job, not a simultaneous snapshot or transaction. Policies are deterministic and causal; no randomized expected-cost guarantee is claimed.

Each ready job has a pure, opaque, whole kernel receiving its own-input identity and exact completed predecessor outputs. Every execution performs its fixed work $w_i > 0$, whether inside or outside protection. Partial/incremental evaluation, preparation before readiness and discharge by external work are excluded in this interface. A preparation is tied to its own-input identity and predecessor tuple. Captured outputs cannot be reconstructed by later reads of the mutable map.

States, histories and turns. A state $(x, \mathcal{M}, \mathcal{R}, \ell)$ contains live own-input identities, completed immutable outputs, a finite snapshot/record store, and program locals. A record carries its job, captured identity, predecessor tuple and computed result. Records may be retained, copied and selected, but not fabricated or retagged without computation. In Table 2, U means no selected record, C a selected possibly stale record, and D permanent completion. A cheap failure clears selection but preserves retained copies. At every finite history each unfinished job admits a legal identity never previously used for that job. Writes preserve completed outputs.

Visible histories list actions, returned values and completion events. Cached completion exposes its identity-comparison Boolean alongside the completed output; pure recomputation may return the same canonical result. Completion publishes its saved output over the live own-key entry. A program chooses a ready job/mode or an unprotected inspection, preparation or local action. The environment then selects a finite word of legal writes: the operation's *gate*, before its indivisible protected part for a callback. Job/mode is fixed before the gate; later record selection may use only the prior store. A cached mismatch recomputes and completes within the same counted call; its exact comparison has no spurious mismatch. Completing comparison/computation/publication is indivisible to writes, and no guard survives return. Inspection honestly returns the live entry.

Preparation computes from a captured immutable own snapshot and exact completed parents; old raw snapshots may predate the counted writer interval. Its pure body has no live read or interaction. At completed-operation boundaries, contract it with its returned record and full work, replaying writes since capture before the next operation. Section 7's relation retains pending writes and does not place a result or full charge at an unfinished concrete prefix.

The common start precedes all computed initialization: every kernel execution contributes to W, including discarded preparations and computation from old raw snapshots. No fixed initialization charge is mandatory. L counts its protected part. Q counts validation/publication/completion calls, including failures; preparations and writers do not consume it. Waiting, common call overhead and nonkernel allocation are excluded. Neither clocks nor work counters are implicitly observed. Earlier call-charge and partial-repair objectives remain distinct in the supplement. Write $\Omega_k(T)$ for the sum of the largest $\min(k, |T|)$ works and $\Omega(T)$ for the total; omitted sets mean J, and $\Omega_0 = 0$. These aggregate/allocation objects are inherited [13, 14].

Table 1. Resources and information contracts used in the results.

Symbol	Meaning
$Q \le n + r$	At most r noncompleting calls beyond the n mandatory completions.
W, L	All foreground kernel work and its protected part.
B	Upper bound on external writes; an input only for informed policies.
Ω, Ω_k	Total work and sum of the $\min(k, n)$ largest works.
$a_1 \le \cdots \le a_n, A_k$	Sorted works and sum of the k smallest works, $A_0 = 0$.
$\mathcal{P}_3(B, r)$	Policies requiring the call cap only against at most B writes.
$\mathcal{U}_3(r)$	One budget-unaware policy requiring the cap at every finite budget.
$\mathcal{V}_r$	Policies in $\mathcal{U}_3(r)$ also attaining optimal L at every budget.

Table 2. Whole-kernel operations. Every validation or callback adds one to Q; preparation adds none. Entries show successor state and added (W, L). Each callback holds protection only within that operation.

Operation	Start	Match	Mismatch
Prepare	U	C, $(w_i, 0)$	—
Cheap publication	C	D, $(0, 0)$	U, $(0, 0)$
Fresh callback	U	D, (w_i, w_i)	—
Cached callback	C	D, $(0, 0)$	D, (w_i, w_i)

A cheap call publishes only on match and exposes success/failure. Conditional publication and a validation-only callback discarding its raw result implement this transition (Section 7). Two modes offer cheap/fresh operations; three add cached completion. The label "cheap" concerns kernel work. Failed validation leaves the entry unchanged. Free inspections/local record operations neither publish nor retain a guard. Direct acquire/release, retained protection, cross-job callbacks and outside completion are excluded, so this is not an arbitrary-lock-API lower bound. Selected operations must terminate. Infinite plays, including endless free actions, have infinite objective.

Strategies and admissibility. An informed strategy is a deterministic function $P(B, h)$; an unaware strategy is $P(h)$. Let $\mathcal{E}_b$ contain gate strategies using at most b writes along every play. For mode family j, $\mathcal{P}_j(B, r)$ consists of strategies completing with $Q \le n + r$ against every $E \in \mathcal{E}_B$. By contrast, $\mathcal{U}_j(r)$ consists of one B-independent strategy satisfying that cap for every $b \ge 0$ and $E \in \mathcal{E}_b$. The gate adversary is adaptive. A fixed deterministic adverse play can be scripted at gates; this does not establish a wall-clock-oblivious or randomized value. Every finite-write play against a fixed program is preserved by capping its environment at that play's write count. Worst cost at budget B is $\sup_{E \in \mathcal{E}_B} L(P, E)$; the two minima use different admissible classes.

Lemma 1 (Fresh-write and finite-adversary facts). *In Section 2's game, a fresh own-key write at a selected i-operation's gate invalidates every retained i-record. Any concrete prefix has both a no-write and a fresh-write continuation whenever that extra write respects the environment's budget. An adversary bounded by b writes on every play forces any $P \in \mathcal{P}_j(b, r)$ to finish every job, regardless of free inspections or record retention.*

Proof. A fresh identity differs from every retained target identity. Copying/selection cannot change that, and all observations and job/mode choices agree until the gate. The bounded adversary belongs to $\mathcal{E}_b$; leaving any job unfinished violates admissibility even with infinitely many free actions. Thus every job completes through Table 2. No unbounded adversary is truncated. □

Lemma 2 (Work charged by useful comparison writes). *From the charged common start, consider a complete execution in which the adversary writes only fresh own-key identities at comparisons whose zero-write continuation would match. Let H be the multiset of resulting cheap failures and D the set of resulting cached mismatches. Then*

$$W \;\ge\; \Omega + \sum_{h \in H} w_{i(h)} + \sum_{i \in D} w_i.$$

Already-stale failures may consume calls, but need no extra charge in this inequality.

Proof. Charge the computed record selected by the *zero-write* continuation, even if the actual failure branch selects an older one. The fresh write invalidates every prior target record. Before completion, neither failed cheap calls nor other foreground actions restore an earlier live identity;

the restricted adversary never restores one. Each later useful comparison therefore charges a different computed identity. Final completion needs a further distinct execution: protected work or a matching preparation for the final identity. A cached mismatch pays both its pre-gate preparation and recomputation. Different jobs cannot share charges. This proves the bound with old snapshots, early computation and retained records. The charge is global; a suffix may have prepaid records. □

3 Universal Call and Protected-Work Guarantees

For unfinished jobs S, let $\mathcal{A}(S) = \{i \in S : \mathrm{Pred}(i) \cap S = \varnothing\}$ denote the ready jobs.

Theorem 1 (Protected work under a call cap). *In Section 2's interface, fix integers $B, r \geq 0$. Among programs supplied with B that complete all jobs with $Q \leq n + r$ in every environment with at most B writes, the minimum worst protected work is*

$$L_3^*(B,r) = \Omega_{\min((B-r)_+,n)}, \qquad L_2^*(B,r) = \begin{cases} 0, & B \leq r, \\ \Omega, & B > r, \end{cases}$$

with and without cached completion, respectively. Any DAG is permitted.

Proof. At most r calls may fail to complete. Freshly prepared cheap attempts have disjoint read-to-comparison intervals; each failure needs a distinct write. If $B \leq r$ they give $Q \leq n + B$, $L = 0$. Otherwise switch after r failures to cached completion: at most $B - r$ writes remain, giving the stated bound. All-protected gives the two-mode upper bound.

For two modes when $B > r$, reject cheap calls with fresh writes, stopping unconditionally after $r + 1 \leq B$ rejections. The call cap forbids exhausting this budget, so every job completes under protection.

For three modes, target the $k = \min(B - r, n) > 0$ heaviest jobs. Count noncompleting target comparisons f and completed targets p, initially zero. Write a fresh identity only at target comparison gates while $f \leq r$ and $p < k$; stop permanently at $f = r + 1$ or $p = k$. Counters follow actual outcomes, including fresh completions. While active, cheap target calls fail, cached calls recompute, and fresh completion protects its body without a write.

At operation boundaries the write count satisfies $v \leq f + p$: each write caused a failure or cached completion. Before another write, $v + 1 \leq f + p + 1 \leq r + k \leq B$. This is unconditionally an $\mathcal{E}_B$ strategy, even on plays violating the call cap.

Admissibility excludes $f = r + 1$: those failures plus n mandatory completions violate the cap. Lemma 1 forces every target to complete, fresh or by cached mismatch, protecting its body. Hence $p = k$ and $L \geq \Omega_k$. Free inspections, stale selections and retained copies cannot evade this. Predecessors contribute their own mandatory calls; target predecessors are counted once in p. □

Theorem 2 (One policy for every writer budget). *Let $\mathcal{U}_j(r)$ contain the j-mode programs that receive no B and complete with $Q \leq n + r$ in every finite-write environment. Set $U_j(B,r) = \inf_{P \in \mathcal{U}_j(r)} \sup_{\#\mathrm{writes} \leq B} L(P,E)$. Then*

$$U_3(B,r) = L_3^*(B,r), \qquad U_2(B,r) = \Omega\,\mathbf{1}[B \geq r].$$

One policy per mode family attains these values simultaneously for all B and positive works on any DAG, using only readiness and a global cheap-failure counter.

Proof. Switch after r cheap failures to a fresh read/preparation with cached completion (three) or to fresh protected completion (two). This uses at most r extra calls universally. At $B < r$ no protection is needed; at $B = r$ cached completions after switching still match. Theorem 1 supplies the remaining upper and three-mode lower bounds.

For two modes when $B \geq r$, reject every cheap call with a fresh own-key write until r rejections, then stop. Only protected calls complete before this. Afterwards, another rejectable call could fail in a different finite environment, violating the call cap. Thus all jobs complete under protection with at most $r \leq B$ writes. That additional hypothetical write is absent from the lower-bound execution. Retained preparations and inspections cannot prevent a fresh write before comparison. □

For equal works and $B > r$, the protection ratio is $n/\min(B-r, n)$. At $B = r$, informed two-mode policies know no write remains; unaware policies must tolerate another-write extensions. Per-update and shared-batch call allowances also define different contracts. Their formulas/proofs remain in the supplement; source comparisons report the distinct caps.

4 The Total Work Needed for Optimal Protection

The protection optimum alone permits wasteful preparation. We minimize total work under simultaneous and supplied-budget protection contracts, for arbitrary r. The separate suprema $(\sup W, \sup L)$ may have different witnesses; they do not determine $\sup(W + \lambda L)$. All computed initialization remains charged.

4.1 One Policy with Optimal Curves at Every Budget

Let $\mathcal{V}_r$ contain the policies in $\mathcal{U}_3(r)$ satisfying $L(P, E) \leq \Omega_{(b-r)_+}$ for every $b \geq 0$ and $E \in \mathcal{E}_b$. This requires one *simultaneous optimal-protection curve*; it is stronger than universal completion. Write $W_P(B) = \sup_{E \in \mathcal{E}_B} W(P, E)$.

Lemma 3 (Capping a realized play). *For $P \in \mathcal{V}_r$, a finite prefix containing d actual writes has protected work at most $\Omega_{(d-r)_+}$.*

Proof. Continue the prefix without writes. Admissibility forces completion. Cap that environment unconditionally at d: induction preserves each action and realized write, and there are no later writes to remove. The resulting environment belongs to $\mathcal{E}_d$ and preserves the complete play. Its total protected work satisfies the bound; nonnegative costs give the prefix bound. The program need not observe d. □

Theorem 3 (Least total-work curve under simultaneous optimal protection). *For independent jobs with sorted positive works $a_1 \leq \cdots \leq a_n$,*

$$\inf_{P \in \mathcal{V}_r} W_P(B) = F_r(B) := \begin{cases} \Omega + Ba_n, & B \leq r, \\ \Omega + \max\limits_{1 \leq m \leq \min(B-r, n)} \{ra_{n-m+1} + \Omega_m\}, & B > r. \end{cases}$$

One policy attains this entire curve: process jobs in ascending work order, freshly prepare before each cheap call until r cheap failures, and freshly prepare before each cached completion thereafter. It observes neither B nor cached comparison results. Sorting, prefix sums and prefix maxima construct the curve in $O(n \log n)$ arithmetic/comparison operations, representing its initial linear segment analytically.

Proof. *Upper bound.* Each bad comparison needs a write in its distinct preparation-to-comparison interval. There are at most r cheap failures; cached mismatches occur only thereafter, at distinct jobs. Thus $Q \leq n + r$ and $L \leq \Omega_{(B-r)_+}$ at every budget. When $B \leq r$, at most B preparations are lost, each costing at most a_n.

For $B > r$, suppose cached mode starts at rank j and has $m \geq 1$ mismatches. All failed preparations were at ranks at most j and the m mismatched jobs lie in the suffix, so $j \leq n - m + 1$. Duplicate work is at most $ra_j + \Omega_m \leq ra_{n-m+1} + \Omega_m$, with $m \leq B - r$. Without a mismatch it is at most ra_n, below the $m = 1$ term. For each permitted m, match the first $n - m$ jobs, fail cheap r times at the

next job, and mismatch all m cached completions. This attains its term using $r + m$ writes. The analogous maximum-job witness attains $\Omega + Ba_n$ at $B \le r$.

Lower bound for arbitrary programs. Fix $m \le \min(B - r, n)$ and target the m heaviest jobs. Write a fresh own-key identity only at target comparisons whose zero-write continuation would match. Let f count these useful cheap failures, c useful cached mismatches and t all target completions. Stop forever after $f = r + 1$ or $t = m$. At operation boundaries the write count is $d = f + c$, with $c \le t$. Before another write, $d + 1 \le f + t + 1 \le r + m$. This is a globally $(r + m)$-bounded adversary even on inadmissible programs.

An admissible program cannot reach $f = r + 1$, and every target eventually completes. Before all targets finish, no target comparison can match: a useful comparison is invalidated, and a stale one already fails or protects. Thus all targets complete protectively and $L \ge \Omega_m$. Lemma 3, applied at the realized final count d, gives

$$\Omega_m \le L \le \Omega_{(d-r)_+}, \qquad d = f + c \le r + m.$$

Positivity forces $d = r + m$, hence $f = r$ and $c = m$. All target completions are useful cached mismatches; this is a consequence of the contract, not an assumed normal form. Lemma 2 now charges r distinct failed preparations of work at least a_{n-m+1}, all m duplicate target kernels and mandatory Ω. Maximizing over m gives the lower bound.

Early budgets. The $m = 1$ witness just constructed has r useful cheap failures at a maximum-work job, then its useful cached mismatch. Truncate after the first $B \le r$ writes. Its unchanged prefix has destroyed B distinct paid preparations of that unfinished job. Eventual completion needs another distinct kernel, giving $\Omega + Ba_n$ by global charging. At $B = 0$ mandatory work suffices. Lower bounds include retained records, free inspections and paid early preparations after readiness. □

The maximizing m need not be largest: works $(1, 100)$ and $r = 1$ give terms 200 and 102. Optimal W and optimal L may have different maximizing executions. The theorem identifies one least attainable pair of curves, not the work of every optimal-L policy. A relaxed protection requirement defines a different comparison.

Dependency boundary. The lower bound is order-independent. For any DAG, any topological cheap-then-cached policy attains $F_r(B) = \Omega + Bw_{\max}$ at $B \le r + 1$, while retaining optimal L at all budgets. At $r = 0$, all-cached gives $F_0(B) = \Omega + \Omega_B$ for every DAG and budget. Later positive-retry values can differ: the chain $10 \to 1 \to 1$, $r = 1, B = 3$, requires $W = 33$, versus 32 for independent jobs. Two target writes force a cheap failure and cached mismatch at the 10-unit job. Its protected work already equals $\Omega_1 = 10$; the next job cannot be fresh on the no-more-write continuation, nor cheap with no retry left. A third write duplicates that unit: mandatory work 12, a failed 10-unit preparation, a cached 10-unit duplicate and one unit give $12 + 10 + 10 + 1 = 33$. The chain policy attains this value. A least entire curve for arbitrary positive-retry DAGs remains unresolved.

4.2 The Complete Frontier for Independent Jobs

A supplied write bound permits a different policy for each resource contract. Here allow arbitrary retry slack $r \ge 0$. Sort independent jobs by positive work $a_1 \le \cdots \le a_n$ and let $A_k = \sum_{j=1}^{k} a_j$, $A_0 = 0$. Put $h = \min(r, B)$ and $p = B - h = (B - r)_+$.

Theorem 4 (Independent-job frontier). *For independent jobs, the exact* $(\sup W, \sup L)$ *frontier over* $\mathcal{P}_3(B, r)$ *is the nondominated subset of* (Ω, Ω) *and*

$$(\Omega + C_k, \Omega - A_k), \qquad 1 \le k \le n - p,$$
$$C_k = ha_k + A_{k+p-1} - A_{k-1}.$$

For $p = 0$ the prefix difference is zero. If $p \geq n$ only (Ω, Ω) remains; at $B = 0$ the frontier is $(\Omega, 0)$. Sorting and a prefix-sum sweep construct the frontier in $O(n \log n)$ arithmetic/comparison operations.

Proof. For selected k, process jobs in ascending work order. Count matching completions s and cheap failures f. Before each preparation, if $s = k$, finish the suffix fresh; otherwise prepare/cheap when $f < r$ and prepare/cached when $f = r$. Stop when all jobs finish. This policy reads no B after k is selected.

Let m count cached mismatches. Disjoint invalidation intervals give $f + m \leq B$; cached mode requires $f = r$, so $m \leq p$. If all jobs finished before $s = k$, then $n = s + m \leq k - 1 + p < n$, a contradiction. Hence exactly k matching jobs precede the fresh suffix. Their weight is at least A_k, giving $L \leq \Omega - A_k$, attained without writes.

Cheap failures occur before cached mode, at rank $s + 1 \leq k$, and cost at most ha_k. The jth cached mismatch has rank $s + j \leq k + j - 1$. Thus $W - \Omega \leq ha_k + \sum_{j=k}^{k+p-1} a_j = C_k$. To attain it, match the first $k - 1$ jobs, fail cheap h times on job k, then cause p cached mismatches at ranks $k, \ldots, k + p - 1$. The next job $k + p \leq n$ matches and triggers fresh completion. For $p = 0$, job k itself matches after the failures. The call bound is $Q \leq n + h$ within budget and $Q \leq n + r$ against any finite number of writes.

For the lower bound, target ranks $k, \ldots, n$. A globally B-bounded adversary writes only at target comparisons whose zero-write continuation would match. Already-stale cheap calls receive no write; stale cached calls protect without one. If the budget is not exhausted, every target completes protectively, giving $L \geq \Omega - A_{k-1}$. Otherwise let $f \leq h$ writes cause cheap failures; the other $B - f$ cause cached mismatches at distinct targets. Lemma 2 gives

$$W - \Omega \geq fa_k + \sum_{j=k}^{k+B-f-1} a_j \geq C_k.$$

The actual distinct targets ensure this window exists. Increasing f replaces its last weight by a_k and cannot increase it; at $f = h$ it equals C_k. Wasted stale failures consume retry allowance without reducing this bound.

Consequently worst $L < \Omega - A_{k-1}$ forces worst $W \geq \Omega + C_k$. Since C_k is nondecreasing, a cap $M < C_1$ forces all work protected. Otherwise choose the largest k with $C_k \leq M$; the next index's target bound forces $L \geq \Omega - A_k$. At $k = n - p$, Theorem 1 supplies the same lower bound Ω_p. The attaining policies complete the frontier; nondominance removes tied thresholds. At $B = 0$, select $k = n$; at $p \geq n$, mandatory work and the protected-work lower bound give all-fresh. □

One fixed r, k policy attains its stated pair for every $B \leq r + n - k$. At the next budget, matching $k - 1$ jobs, failing cheap r times and mismatching every remaining job gives $L = \Omega - A_{k-1} > \Omega - A_k$. Thus the constant protection guarantee's range is sharp. Selecting k still uses the supplied contract; this is not one optimal-L curve for all unknown budgets.

The supplied and simultaneous-optimality contracts can have different least work, even though both achieve optimal protection. Figure 1 gives their $r = 1$ curves; the supplement derives the zero-retry corner directly from the two theorems.

4.3 When Retry Slack Covers the Write Budget

Proposition 5 (An order-independent frontier without the cached flag). *If $r \geq B > 0$, on* every *DAG the least worst L under $W \leq \Omega + M$ is*

$$\sum_{i:Bw_i > M} w_i.$$

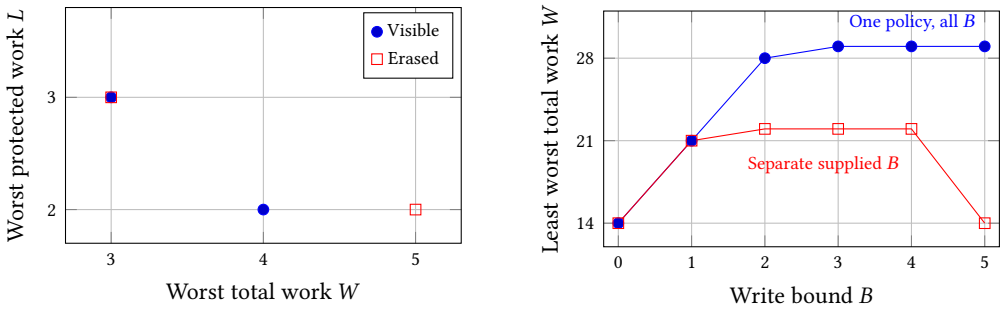

Fig. 1. Independent jobs. Left: exact two-job frontier points for visible and erased cached comparisons; no interpolation is implied. Right: for works $(1, 2, 4, 7)$ and $r = 1$, both classes achieve $L^*(B) = (0, 0, 7, 11, 13, 14)$ for $B = 0, \ldots, 5$, but the upper curve uses one policy meeting all bounds. The supplied-budget optimum can fall as B grows because its optimal-L cap also loosens; it compares different contracts. Values are kernel work, not timings.

Its attaining policy needs only cheap success/failure, even under the output-only cached contract of Section 6.2.

Proof. Choose the k smallest jobs for cheap calls and all others fresh; follow any topological order until B cheap failures, then prepare/cache every unfinished job. Duplicate work is at most Ba_k, and protected work at most $\Omega - A_k$. Repeated writes at a maximum chosen job and the no-write path attain the separate bounds. Theorem 4's target lower bound uses no execution order and specializes to $C_k = Ba_k$. Tied thresholds give the displayed cap form. □

For the 8/16 example, one spare call and supplied $B = 1$ permit $(W, L) = (32, 16)$ and $(40, 0)$ in either edge direction. Enough failures can reveal budget exhaustion through cheap results; the cached comparison need not be exposed. This does not eliminate the observation issue when $r < B$ or when one optimal-L curve is required at every unknown budget.

5 Precedence Determines the One-Write Frontier

Consider supplied $B = 1$, $Q \le n$ and a body-work cap $W \le \Omega + M$. To include protected entry work, charge each guarded completion $g_i \ge 0$, cheap calls zero, and set $\Gamma = \sum_i g_i$ and $P_g = L + \sum_{\text{guarded}} g_i$. The first coordinate here remains body work; Section 8 charges overhead in both coordinates. Define

$$(p_i, \tau_i) = \begin{cases} (g_i + w_i, 0), & w_i > M, \\ (g_i, w_i), & w_i \le M. \end{cases}$$

Call these jobs heavy and light, respectively. For a topological order π, $C_i^p(\pi)$ sums p_j through i.

Theorem 6 (Reduction to precedence-constrained scheduling). *For every DAG, the minimum worst P_g under the supplied-one-write call/body-work caps is*

$$\ell_g(M) = \min_{\pi \text{ topological}} \max_i \{C_i^p(\pi) + \tau_i\}.$$

Lawler's backward minimum-cost sink rule [34] computes an attaining order. Sweeping zero and the distinct work thresholds, then removing dominated pairs, gives the complete W/P_g frontier. Setting $g_i = 0$ gives the body-only W/L result.

Proof. Follow an order π, completing heavy jobs fresh and light jobs by immediate prepare/cache. After the first cached mismatch, prepare/cheap-complete the suffix. This mismatch of a fresh record proves that the only possible write is spent, so every suffix call succeeds. Exactly n calls complete, and only that light job duplicates body work, of weight at most M.

With no mismatch, protected cost is $\sum_i p_i$. A mismatch at light i gives $C_i^p + w_i$, with no later guarded charge. A fresh job's term C_i^p is bounded by $\sum_i p_i$. No writes and each selected own-gate write realize the respective cases, giving the displayed maximum.

For any admitted program, follow a no-write execution. A cheap call would admit a one-write failure and violate $Q \le n$. The guarded completions induce a topological order π. Let F be its fresh or already-stale cached completions; old raw snapshots can be stale without a new write. Every matching cached $i \notin F$ has an own-gate write continuation forcing $W \ge \Omega + w_i$ by Lemma 2, so all heavy jobs belong to F.

The no-write path gives $P_g \ge \Gamma + \sum_F w_i$. Each $i \notin F$ gives a separate one-write witness with cost at least its guarded-charge prefix, its preceding F bodies, and w_i. Take their maximum. Removing an optional light j from F in this lower-bound expression decreases the total and all later fresh prefixes. Its new own term is at most the old no-write total, since all remaining guard/body charges are nonnegative. Repeat until only heavy jobs remain; the expression becomes $\max_i(C_i^p + \tau_i)$. This is algebra on lower bounds, not a replacement of the original program's concrete execution. Minimizing over its possible no-write order proves completeness.

For the inherited scheduling exchange, choose a remaining sink j of minimum τ_j last. If an optimal order ends in sink k and has total processing T, moving j last decreases intervening completion times and gives $T + \tau_j \le T + \tau_k$. Recurse; zero processing times are allowed. Crossing a weight threshold leaves that job's own term unchanged and decreases later prefixes. Intermediate M has actual duplicate cap $\max(\{w_i : w_i \le M\} \cup \{0\})$, so the stated finite sweep suffices. □

The completion-to-scheduling equivalence is the result; Lawler's algorithm is unchanged. A simple implementation takes $O(n^2 + |E|)$ per threshold and $O(n^3 + n|E|)$ for the sweep. One cannot obtain a fully charged total-work frontier by adding Γ to W: the cheap suffix makes the charge path-dependent.

Proposition 7 (When the optimal protection cap permits a saving). *At $g_i = 0$, $B = 1$ and $L \le w_{\max}$, the minimum worst total work is $\Omega + w_{(2)}$ if the maximum is unique and its job is a sink; otherwise it is $\Omega + w_{\max}$. Set $w_{(2)} = 0$ for one job.*

Proof. If $M < w_{(2)}$, at least two heavy jobs have total exceeding $w_{\max}$. If the unique maximum has a successor, that successor's term exceeds $w_{\max}$ whenever $M < w_{\max}$. Tied maxima also force excessive heavy total below this threshold. Conversely, for a unique maximum sink, $M = w_{(2)}$ and placing that job last gives objective $w_{\max}$. At $M = w_{\max}$ all-cached always attains it. □

For $g_i = 0$ and $8 \longrightarrow 16$, cache the small job, then use fresh on match and cached on mismatch. The two paths cost $(24, 16)$ and $(32, 8)$, hence worst $(32, 16)$. Reversing the edge or imposing simultaneous optimal protection requires $(40, 16)$.

6 A Complete Finite Policy Class for Arbitrary Retry Slack

The preceding polynomial regions have direct lower bounds. For other DAG/budget combinations we justify finite policy search against the full class of programs with free inspections, early preparation after readiness, and retained records.

Theorem 8 (Resource domination by immediate-preparation trees). *For every $P \in \mathcal{P}_3(B, r)$ there is a finite fresh/cheap/cached tree T such that*

$$\forall E \in \mathcal{E}_B \; \exists E' \in \mathcal{E}_B : (Q, W, L)(T, E) \preceq (Q, W, L)(P, E').$$

Its depth is at most $n + r$, and it prepares only immediately before its selected cheap/cached node.

Proof. *Extraction.* Restrict the adversary to no writes except an optional fresh own-key write at a comparison whose zero-write continuation matches. Fix a concrete fresh identity at each branch history. A fresh call gives a unary fresh node. A matching cheap call gives completing and fresh-write-failing children; a matching cached call gives two completing children. An already-stale cached call becomes unary fresh, retaining its original zero-write continuation. Delete an already-stale cheap call and retain its failing continuation and fixed local choices in the witness history.

Finite call structure. If the original prefix has u completions, h useful cheap failures and s deleted stale failures, its calls number $u + h + s$; the tree has $u + h$. Its unused failure allowance may increase, but need not be spent. All represented paths have at most $n + r$ nodes. Between them, no-more-write computation and deleted calls must terminate, or the original program would not complete in that finite-write environment. Finite branching and bounded depth therefore yield a finite tree.

Cap the restricted adversary at B and replace a zero-remaining-budget suffix by ready prepare/cached completions. Useful comparisons cost counted calls; the original cap forces completion along every represented path.

Global charging. On a complete path let H be its binary cheap failures and D its binary cached-mismatch jobs. Lemma 2 lower-bounds original work by $\Omega + \sum_H w_i + \sum_D w_i$. Original protected work is at least the sum of the extracted fresh-node works and $\sum_D w_i$; a collapsed stale cached call already protects its kernel. A known-zero-budget suffix substitution can only lower protected work. Deleted stale cheap calls need no separately assigned preparation charge.

Realization and domination. Realize the fixed tree using actual current inputs and exact saved parents, ignoring their values for scheduling. Prepare only at its selected cheap/cached node, never speculatively for a fresh node. Readiness follows the same completed set. Actual failures/mismatches have disjoint preparation-to-comparison intervals and inject into distinct actual writes; irrelevant or identity-reusing writes may remain invisible. Thus a path with $d = |H| + |D|$ bad comparisons requires at least d writes. Exactly d own-gate fresh writes realize that path and its original concrete witness. Define the witness adversary with an unconditional cap d, stopping after that many writes on every history. It is globally d-bounded, rather than merely spending d on one play.

The tree's exact path costs are $Q = n + |H|$, $W = \Omega + \sum_H w_i + \sum_D w_i$, and $L = \sum_{\text{fresh}} w_i + \sum_D w_i$. They are no greater than the original witness's. Any actual tree execution with at most B writes is therefore matched to an original environment in $\mathcal{E}_d \subseteq \mathcal{E}_B$. Correct outputs follow from the operations' semantics, not from reproducing the original program's concrete values under the same environment. □

6.1 Exact Pareto Recurrence

Let $\mathcal{F}(S, b, a)$ contain nondominated suffix-tree bounds (e, l), where S is unfinished, b is a conservative remaining write allowance, a remaining noncompletions, $e = \max W - \Omega(S)$ and $l = \max L$. Set

$$\mathcal{F}(\varnothing, b, a) = \mathcal{F}(S, 0, a) = \{(0, 0)\}.$$

For ready i, write $R = S \setminus \{i\}$ and take the nondominated union of

$$\begin{aligned} &\text{fresh:} && (e, w_i + l), \quad (e, l) \in \mathcal{F}(R, b, a); \\ &\text{cached:} && (\max\{e_0, w_i + e_1\},\ \max\{l_0, w_i + l_1\}); \\ &\text{cheap, } a > 0: && (\max\{e_0, w_i + e_1\},\ \max\{l_0, l_1\}). \end{aligned}$$

Both comparisons choose their matching child in $\mathcal{F}(R, b, a)$. Cached chooses its other child in $\mathcal{F}(R, b-1, a)$; cheap chooses it in $\mathcal{F}(S, b-1, a-1)$. Children are selected independently.

Lemma 4 (Sufficient state for a suffix tree). *The recurrence gives exactly the nondominated immediate-preparation tree bounds. Here b counts the remaining allowed* bad branches*; it need not equal actual unobserved writes remaining.*

Proof. A path with d bad branches needs at least d distinct invalidating writes. Conversely, exactly one fresh own-key write at each bad gate and no others realizes that path. Hence worst costs with at most b writes equal maxima over paths with at most b bad branches. Inert writes or multiple writes at one gate add no new path.

Induct on $|S| + a$. At $S = \varnothing$ there is no cost. At $b = 0$, prepare/cached completes every ready job with no duplicate or protected work, dominating other suffixes. Otherwise a fresh node has one child on R. A comparison has a matching child with allowance b and a bad child with allowance $b - 1$; a cheap failure additionally consumes one of a retries and leaves S unchanged. Each child decreases the induction rank. Readiness depends only on S; under immediate preparation, costs depend only on the selected job/mode and outcome. These facts yield exactly the displayed charges.

The two children describe mutually exclusive observations. The controller may choose either continuation independently in advance; the adversary follows only one. It therefore receives no split budget across children. Taking a maximum separately in each resource is both necessary and sufficient for the two worst-case caps. Conversely every tree has precisely this root/child decomposition. Monotonicity of each charge/max expression allows a dominated child to be replaced by a dominating one. Emitted policies retain witness nodes; original histories with the same (S, b, a) need not be equivalent. Theorem 8 supplies full-program completeness. □

Pareto backward induction itself is inherited.

There are at most $(r+1)(\min(B, n+r)+1)2^n$ states, with possibly exponential frontier sizes. The independent-job formula, enough-retry DAG formula and zero-retry one-write Lawler reduction identify polynomial regions; a general fixed-order theorem is not claimed.

The supplement also extracts one budget-independent tree for a universal program by retaining every branch up to depth $n + r$. Theorem 3 settles its least total-work curve for independent jobs; general DAGs may still require a Pareto set of curves.

6.2 What the Cached Comparison Reveals

Consider an erased observation contract. Every invocation returns the same immutable per-job object z_i, which completion publishes over its live entry. The cached primitive's *entire visible transition* exposes only that output and completion: no compared input, Boolean, comparison-dependent record selection or callback-local side effect survives. Kernel internals, work counters, allocation IDs and clocks are hidden. Free outside inspections and old snapshots remain allowed. Constant output alone is insufficient if the callback saves the current input elsewhere.

Proposition 9 (A strict observation boundary). *For independent works $(1, 2)$, supplied $B = 1$ and $Q \le 2$, the visible-comparison frontier is $\{(3, 3), (4, 2)\}$; erasure changes it to $\{(3, 3), (5, 2)\}$.*

Proof. All-fresh attains $(3, 3)$. For $(4, 2)$, cache the unit job; on match complete the other fresh, and on mismatch prepare/cache it. The latter branch certifies the sole write. Theorem 4 gives the visible frontier.

For erasure, follow any admitted program's no-write run. Cheap would permit an extra failed call, so both completions are guarded. Partition them into matching cached C and fresh/already-stale cached F. If F is nonempty, either both bodies are already protected or a single own-gate write at the remaining matching cached job protects its body too. Erasure restores the same visible state after that completion, preserving subsequent inspections, choices and protected F bodies. Thus worst $L \geq 3$ is unavoidable.

If $L < 3$, both jobs must instead be matching cached on that path. One fresh write at the 2-unit job forces $W \geq 3 + 2 = 5$ and $L \geq 2$, by Lemma 2. Immediate prepare/cache for both attains $(5, 2)$: at most one comparison can mismatch. These cases exhaust programs and prove the erased frontier. □

Figure 1 shows this separation. The supplement gives the all-DAG zero-retry erased frontier and a positive-retry DAG separation; the general positive-retry case remains open.

7 Actual External Writes in Java

Use Java 17 `ConcurrentHashMap`, one controller, permanent keys, deeply immutable non-null values, identity equality and never-reused own-key write identities. Kernels read captured own inputs and saved completed parents; kernels/writers preserve tags/shapes. Key methods are stable; methods terminate without side effects or recursive updates. Replacement may call `equals` [19, 20]. Normal progress is a premise.

Cheap wraps `replace` or validation-only `compute`, exposing success/failure. Cached exposes its saved output and callback-local comparison Boolean; fresh always computes. Observations retain opaque-reference equality, payloads, wrapper results, saved parents and selector state. Numeric IDs, clocks, counters, raw cheap-mismatch values and diagnostics are hidden.

Correspondence. An injective reference map η preserves aliases across live entries, pending writes, saved results and retained handles. Set $\mathcal{M}(i) = \eta(\mathsf{done}[i])$ from the saved output. The store maps exactly available records to $(i, \eta(s), \mathcal{M}|_{\mathrm{Pred}(i)}, \eta(z))$; discarded results are unavailable. Locals map to ℓ.

Let Δ queue committed external assignments deferred in normalization. At boundaries,

$$\mathrm{decode}_\eta(H.\mathsf{map}) = \mathrm{apply}(\Delta, x), \qquad b_H = b_x + |\Delta|.$$

Here b counts publications, not diagnostic increments. Decoding follows forwarding nodes; Δ replays assignments, not callbacks. Atomic `compute` invokes its callback once [18]; the pinned implementation serializes same-key updates in list/tree bins [20]. Wrappers return saved operation results even if the live entry is newer. Completion saves that output, never a reread.

Theorem 10 (Resource bounds for actual writes). *Under these assumptions, the call/body-work results ($g_i = 0$) of Sections 3–6 hold for actual external writes, under each result's observation contract and with separate or shared bins. Completed-operation traces preserve results and Q, W, L. This is not a mechanically verified JDK/JMM theorem.*

Proof. Group each completed pure kernel with its operands, result and work. Defer writes since capture in Δ and flush before the next operation, including inspection. Unfinished instructions are not zero-cost stutters. Post-publication/pre-return writes also enter Δ: saved outputs fix $\mathcal{M}$ while the map may be newer. Flush terminal queues without adding writes. Resize/tree conversion copies references; concurrent publications remain writes.

Upper policies associate bad comparisons with distinct committed writes through disjoint fresh preparation-to-comparison intervals. Progress transfers their bounds. For lower bounds, suspend after job/mode/store selection, before API entry. A terminating own-key writer installs a fresh reference without a foreground monitor held, even with colliding bins, invalidating all retained target records. Bounded target adversaries preserve write caps, mandatory completions and global charges. Further finite continuations establish universal admissibility. Saved parents preserve readiness; callback Booleans implement observable branches. □

Earlier kernels perform $8w_i$ counted iterations; newer wrappers use w_i units. Waiting, allocation, traversal and writers are outside W, L. Counters are checker-only; the finite matrix excludes concurrent resize. Full source locators and correspondence details are in the supplement.

8 Including Protected Entry and Comparison Work

The following charges are supplied scalar resource costs, not calibrated native times. First retain W as body work and P_g from Section 5. The guard ends at return and cannot be shared across jobs.

Proposition 11 (Entry charges separate the boundary budget). *At $B = r$, the informed optimum of P_g is 0, while the least worst value of an unaware policy is Γ on every DAG.*

Proof. The informed policy completes cheaply. An unaware policy uses cheap until r failures, then immediate prepare/cache; at $B = r$ no further mismatch is possible, so $P_g \le \Gamma$.

For the lower bound, reject cheap calls, writing only when their zero-write continuation matches. Impose an unconditional cap of r writes and stop at r cheap failures. No cheap call can then occur: its stale failure, or one more write in another finite environment, contradicts universal $Q \le n + r$. Thus no cheap call succeeds and every job pays its guard charge. Finite-write admissibility forces completion even with free inspections and retained records. □

The supplement proves the full optimal unaware curve $\Gamma\mathbf{1}[B \ge r] + \Omega_{(B-r)_+}$. Entry charges therefore alter even the zero-body-protection case.

8.1 A Complete Frontier with Charges in Both Coordinates

Now add a nonnegative cost c_i for every cached comparison, whether it matches or mismatches. Fresh completion pays g_i but not c_i. Define

$$W_t = W + \sum_{\text{guarded}} g_i + \sum_{\text{cached}} c_i, \qquad P_t = L + \sum_{\text{guarded}} g_i + \sum_{\text{cached}} c_i.$$

Cheap calls have zero added charge here. Cached mismatch remains observable and completes inside the same call. Costs reveal no new observations and allow no skipped comparisons, batched completion or guard amortization.

For a topological order π and cached subset C, set $F = J \setminus C$, $h_i = g_i + c_i\mathbf{1}[i \in C]$, and $H = \sum_i h_i$. Let H_i sum h_j through i in π and F_i sum the works of F jobs strictly before i. Empty maxima below are 0.

Theorem 12 (Fully charged one-write representation). *At supplied $B = 1$ and $Q \le n$, the complete frontier is the nondominated set over all (π, C) of*

$$\mathrm{W}(\pi, C) = \Omega + \max\{H, \max_{i \in C}(w_i + H_i)\},$$
$$\mathrm{P}(\pi, C) = \max\{H + \Omega(F), \max_{i \in C}(w_i + H_i + F_i)\}.$$

An order, a mode bit per job and a first-mismatch flag specify an attaining policy.

Proof. Follow π, using fresh on F and immediate prepare/cache on C until the first mismatch; then prepare/cheap-complete the suffix. A fresh JIT mismatch certifies the sole write, so all suffix calls succeed. Without a mismatch, the costs are $(\Omega + H, H + \Omega(F))$. A first mismatch at i gives $(\Omega + w_i + H_i, w_i + H_i + F_i)$. No writes and one selected own-gate write realize these cases, whose separate maxima are the formulas.

For an arbitrary admitted program, fix a no-write execution. Cheap is impossible: a fresh gate write would add a failure to n mandatory completions. Guarded completions induce π; take matching cached jobs as C and fresh/already-stale cached jobs as F. Dropping stale cached comparison/preparation charges only weakens the lower bound. This execution costs at least $(\Omega + H, H + \Omega(F))$.

For each $i \in C$, preserve the prefix, write one fresh own-key identity at its gate, then stop. The write invalidates all retained target records. Lemma 2 forces body work at least $\Omega + w_i$, including paid early preparation. The incurred guarded/comparison prefix is at least H_i; protected bodies include $F_i + w_i$. Thus the alternative execution costs at least $(\Omega + w_i + H_i, w_i + H_i + F_i)$. Each environment has an unconditional one-write cap. The program's separate suprema dominate all these witnesses, proving domination by the displayed pair. The attaining policy completes the proof for input-dependent choices and retained records. □

Proposition 13 (Decision and output-size boundary). *For explicit DAGs and binary integer costs, deciding joint W_t/P_t caps in Theorem 12 is NP-complete, already on independent jobs. With N jobs, an exact frontier can have 2^{N-1} points. These statements concern the added-cost class.*

Proof. An order and mask certify both caps in polynomial time. For hardness, take positive subset-sum integers $a_1, \ldots, a_m$ and target T [1]. Give each original job $w_i = 2a_i, c_i = a_i, g_i = 1$. Add an independent dummy z with $w_z = c_z = 1$ and $g_z = G = \max_i w_i$. There are $N = m + 1$ jobs and the call cap is N.

Let $D = \Omega + \Gamma$. For any original subset C, run its cached/fresh choices, then z fresh last. Since $H - H_i \geq G \geq w_i$ at every original cached job, the no-write costs dominate both mismatch costs. Its exact pair is

$$(D + \textstyle\sum_C a_i,\ D - \sum_C a_i).$$

Conversely, extract any program's no-write matching subset. Its no-write pair is a lower bound. Removing a cached dummy decreases total cost by 1 and leaves protected cost unchanged, since $c_z = w_z$. The remaining original subset therefore satisfies the displayed lower bound. Caps $(D+T, D-T)$ hold exactly when $\sum_C a_i = T$. The construction is polynomial in binary input length.

For output size, set $a_i = 2^{i-1}$. All 2^m distinct sums give mutually nondominated pairs $(D+s, D-s)$, with $O(m^2)$ input bits. The general NP statement does not prove strong hardness or an optimal exponential running time. □

In this dummy family, zero original comparison charges give one point $(D, \Gamma + 1)$; equal positive charges give at most $m+1$ cardinality candidates. Its heterogeneous case inherits a pseudopolynomial subset DP. General charged strong/weak hardness and original free-comparison hardness remain open. The dummy guard is a mathematical construction, not a native measurement.

9 Formal Corroboration and Operational Evaluation

RQ1 checks formulas and the operation contract; RQ2 tests a Linux upper refinement; RQ3 examines Roslyn preservation and cost limits. Fixed inputs precede execution; reanalyses are labelled post-outcome. All failures, timeouts, invalids and corrections remain archived.

An Apple M2/24 GiB runs Python 3.10.9/3.12.14 and OpenJDK 17.0.19; the cited map version was inspected, not rebuilt. Hashes bind inputs, code, commands and traces. Codex assisted design, checkers, sources, proofs, experiments, analysis and writing; Claude provided separate proof/direction critiques and manuscript reviews. All measurements are archived author executions, without human studies or independent certification.

An input fixes graph, works and prices; a root adds budget/retries. Paths are complete branch sequences; executions are native invocations. Pairs compare policies/wrappers on one input; groups collect input variants; coordinates add evaluation budgets. Table 3 separates overlapping studies.

9.1 RQ1: Bounded Corroboration of the Proofs

Table 3. Bounded checks of the retained claims. The studies overlap in their source inputs; their denominators are reported separately. Finite agreement is not an all-program proof.

Obligation	Evidence and comparison
General call/L values	Direct minimax versus formulas: 265,629 agreeing roots from 5,421 DAG/weight inputs.
General retry frontier	Immediate and retained-preparation finite models: 96,795 agreeing cases; $n \le 4$, works $\{1, 2, 4\}$, $r = 0, 1, 2$.
Polynomial formulas	Retrospective: 1,998 independent and 32,526 enough-retry DAG cases (720 overlap). Prospective: 13,850 agreeing roots and 540,230 independently interpreted paths.
Unknown-budget curves	647 fixed independent roots and 34,814 interpreted paths; exact least curves agree, zero mismatches. Earlier 713 general roots remain.
DAG/observation boundaries	36 scope roots, zero mismatches; four erasure roots, 8,672 trees and 102,122 paths. All 8,668 protection violations retained.
Zero-retry regressions	26,844 Pareto cases; 18,390 scheduling cases; 2,004 independent corners; 237 simultaneous curves; 2,367 output-only cases. All agree with their stated comparators.
Native policies	Separate matrices of 2,076 zero-retry, 12,224 supplied-budget and 20,384 unknown-budget executions. Zero output/resource mismatches in each.
Flag-ignoring controls	Four of 52 violate L as predicted; all other 48 are retained.

The prospective general-retry study uses 461 multisets, $n \le 6$, works $\{1, 2, 4, 7, 11\}$, $r \le 3$, $B \le n+r$. All 9,240 first-excluded-budget controls violate L, preserving Q; 950 output-only cases have positive Boolean value.

Actual writers exercise both wrappers, colliding/separate bins and canonical/allocating outputs. All 6,112 supplied-budget and 10,192 unaware wrapper pairs agree; 4,336/4,092 executions verify tree bins. The unaware selector sees no B, cached flag or diagnostics. All 512 groups/2,464 coordinates attain their curves; 12 corruptions reject. Unissued gates remain explicit. Entry-charge checks cover 26,874 scalar roots, 1,809 order cases and 86,376 paths. The generalized scheduling reduction agrees on 28,720 thresholds from 8,736 inputs. The fully charged one-write study fixes 14,696 inputs, 760,536 order/mask policies and 850,869 interpreted paths; all declared comparisons agree. All 1,662 subset embeddings also agree in their declared methods, with 1,646 unrestricted mode-DP comparisons and 124,572 witness paths; six controls detect their fixed alterations. These populations overlap earlier inputs.

Earlier aware/unaware records (9,314/17,714), six scaling timeouts, retained-guard/simple-rule counterexamples and cost fits successful in only two of eight cases remain distinct. The supplement

retains the controlled Java blocking probe, all 24 timing cells and reversed disjoint-bin results; none establishes natural-arrival performance.

9.2 RQ2: A Native Traversal and Its Guard Boundary

Linux v6.12 `dentry_path_raw` first walks mutable dentry parents/names under RCU, validates `rename_lock`'s sequence, and on failure rebuilds under its reader-exclusive lock [7]. Pathname lock contention motivated the upstream optimistic fallback [9]. Our KUnit-only variant prepares a second traversal with a *new even* sequence, then acquires the lock and validates that saved sequence: reuse on match, reset/rebuild on mismatch. Every traversed component must be initialized, with no concurrent initialization; a valid reference alone is insufficient. Initial dentry/name publication must precede parallel reading; subsequent relevant changes must participate in the global sequence, without wrap. This covers the initialized `d_exchange` fixtures, not every filesystem: `d_mark_tmpfile` changes names without that sequence during initialization [8]. Mount traversal and `getcwd` are excluded.

Assume a private valid buffer with $1 \le$ `capacity` $\le$ `INT_MAX`, nonnegative observed lengths fitting `int`, valid dentry/RCU lifetimes, and successful stable-name copies. Let $C =$ `capacity` $- 1$ after NUL. In the original factored loop, attempted names v, requested copy bytes c, fault-fill bytes f and successful slashes s satisfy $v \le s + 1$, $f \le c$ and $c + s \le C$. Thus body events $T = 1 + v + c + f + s \le 2C + 2$. For first/second validation failures X, Y, fresh even capture gives $X + XY \le \min(B, 2)$ for actual global writer sections, including unrelated renames. Count one Q unit per lockless validation or guarded validation/completion block, not per native API call. The candidate has $Q \le 2$, $T_{\text{total}} \le (1 + \min(B, 2))(2C + 2)$ and $L_T = 0$ for $B \le 1$. This is a single-job upper refinement: private-buffer return does not instantiate the full publication/observation contract. Guard/RCU instructions, waiting and postambles are separate; the capacity-4096 ceiling 8192 is loose on short paths.

Actual ARM64 Linux boots in QEMU with two CPUs, lockdep and atomic-sleep checking. A pinned writer uses native `lock_rename`/`d_exchange`; endpoint gates are outside RCU and the mid-parent gate spins without sleeping. The fixed second matrix has eight path families, six capacities and eight schedules across five policies: 1,920 valid rows (948 paths/972 expected overflows), plus four detected defects. An independent fixture oracle checks full strings, offsets, canaries and post-return persistence against quiescent upstream. All rows and 2,878 body records agree offline, without lock/RCU diagnostics. A third execution brackets all 1,924 invocations with even nonwrapping global sequences: its 1,763 writer sections equal the designated counts in every row, and all prior output/body records agree. Returning unvalidated overflow, failing to restore the buffer or ignoring final validation breaks output. Reusing the first sequence preserves output but violates the protection bound at $B = 1$. No copy-fault/allocation failure is injected; native-time performance is unmeasured.

Table 4. Source event counts at the scripted gates. Initial path is `/driver/cache/entry/leaf`; first exchange moves it to `/alternate`, second restores it. G counts reader-exclusive acquisitions.

	One write				Two writes			
Policy	T	L_T	Q	G	T	L_T	Q	G
Upstream	41	12	2	1	58	29	2	1
Cached	41	0	2	1	70	29	2	1
Two cheap, then locked	41	0	2	0	70	29	3	1

All 384 cached/two-cheap pairs agree on output and body vectors; 138 first-invalid/second-valid pairs avoid the guard, while 144 two-failure pairs need another logical block. The cap comparison depends on its unit. One public `dentry_path_raw` invocation remains one under every policy. Counting six named primitives—RCU enter/exit, sequence begin/retry, and exclusive lock/unlock—gives $4R + 2G$, where R counts outside traversals. The cached/two-cheap counts are 4/4, 10/8, 10/10 for zero, one and two observed failures: both universal maxima are 10, despite logical caps 2/3. These are source call counts, excluding spinning and underlying instructions. No caller-required $Q = 2$ SLA was found. A stronger native name-snapshot comparator copies inline names or retains external-name references under protection, then materializes outside. Its allocation, dentry locks, capture steps and refcounts are separate from L_T; five 64-slot overflows retain the discarded capture work before protected fallback. Its body-only zero is not zero total protected work or a lower bound for larger workspaces. All adverse results and the first 1,536-row matrix remain; the populations overlap.

9.3 RQ3: Preservation and Cost Limits in Roslyn

Roslyn's `Workspace.cs` at `6c4a46a3` transforms an immutable solution outside a semaphore, validates identity inside and retries [10, 11]. Our `SourceText`-only patch switches after r shared failures to fresh (two-mode) or cached completion (three-mode). A stronger rebase reuses guarded document contents but still requires a protected merge. Lazy work, allocation, version generation and cache-dependent merges prevent fixed-work transfer. No upstream numerical cap or SLA was found.

All 236 authored cases agree on 2,190 events, 1,253 publications and 2,653 snapshots; 12 corruptions reject. In 96 compiler processes, 2,556 project projections match declared expectations for references, parse options and linked text under cold/warm caches; eight corruptions reject. The original same-text deadlock and four missing-document timeouts remain archived; corrected patches pass 14 error cases and reentry controls. These selected checks use Roslyn on both sides. Custom loaders, changed-text reentry and general concurrent equivalence remain outside scope.

Arrival tests compare shared/per-update caps, two/three modes and both baselines. Eight balanced blocks serve 32 updates and 64 FIFO writers each at 0, 10, 100, 1000 μs. All 2,560 measured and 512 warm-up units pass source/count checks; 12 corruptions reject. At $r = 2$, local/shared caps allow 96/34 calls, precluding equal-cap comparison.

Shared three/two foreground-time ratios at 0/10 μs are 1.72 [1.40,2.11] and 1.58 [1.17,2.16], favoring two modes. Slower-arrival and all writer-response intervals include 1. Geometric means use eight paired block ratios and descriptive 95% intervals from 10,000 fixed block-bootstrap samples without multiplicity correction. Baseline ratios 0.83–1.08 all include 1; instrumentation overhead is unidentified. Authored arrivals, eight blocks and one host limit inference; the supplement retains all mixed results.

10 Related Work and Research Lineage

Fallback and contention. Kung and Robinson [6], Section 3.3, retain a semaphore during re-execution. Bounded lock-elision/RPC retries are established [26, 27, 31, 32]; HTM aborts need not consume writes [28], and RPC can fail. Welc et al. [33] preserve transaction prefixes during irrevocable transition. Attiya et al. [24] analyze unaware contention management; Guerraoui et al. [4] bound progress and makespan; Alistarh et al. [25] optimize abort timing. Switching and retry tuning predate ours [5, 41, 42]. These progress/time contracts differ from persistent-job W/L caps.

Synthesis. Černý et al. [15] synthesize retries, guards and caches for weighted limit-average safety (PSPACE by erratum); later work optimizes protection/profile costs [16] or preemption-safe

locks [17]. Global-budget/information-state minimax DP is inherited [12, 21, 29]. General games encode finite instances. Our results establish all-program resource domination and exact polynomial regions; neither backward induction nor Pareto composition is new.

Scheduling with testing. At $r = 0$, fresh completion is untested execution of cost $u_i = w_i$; cached completion tests for $t_i = w_i$, then executes for $p_i \in \{0, w_i\}$. Thus W counts testing plus execution, and L execution alone. A deterministic tree's mismatch set can be fixed as hidden bad-job labels, and own-gate writes realize those labels. Dynamic writing alone does not distinguish this canonical game.

Dürr et al. [35] introduce adversarial testing; Buld and Schulz [38] and Albers and Eckl [39] allow unequal test times/upper bounds, for competitive time objectives. Damerius et al. [36] separate testing expenditure and execution with a *policy-side* budget. Our B limits adverse outcomes; preparations remain allowed at $B = 0$. Dufossé et al. [37] study minimax testing trees with durations revealed even for untested execution; polynomial adaptive optimality assumes their two-phase conjecture. Albers and Janke [40] bound failing jobs for irrevocable parallel-machine assignment. Our specialization returns causal completion policies for separate absolute W/L caps and retry slack. The informed comparator knows only B.

Precedence and reuse. Lawler's rule [34] is unchanged; Theorem 6 proves the completion-to-scheduling equivalence. Universal restart [43] instead concerns independent Las Vegas runs and expected time. Build systems [2], self-adjusting computation [3] and demand-driven incrementality [30] reuse unaffected work, potentially violating the whole-kernel premise. Full source comparisons and partial-repair results remain in the supplement.

11 Limitations and Threats to Validity

The interface excludes replenishment, structural change, exceptions, external completion and retained guards. Under an earlier scalar objective, retained guards improve 5/384 inputs (242 versus 243) and one intermediate state (45 versus 43); these do not evaluate the present W/L frontier.

Determinism, positive fixed work and full charging are premises. General charged strong/weak hardness, free-comparison complexity and least unaware curves for positive-retry DAGs remain unresolved. IDs/timing change observations; supplied B is a premise and Roslyn costs remain uncalibrated.

Instrumentation and cache/JIT effects limit timings; human benefit is unmeasured. Earlier scalar gains occur in 2/96 cells, cost fits in 2/8, and all-budget compilation can cost more than requested-value computation. In all 16 Valkey digest cells, cooperating writer maintenance beats reader repair in time/transfer, with mixed maximum command times. This stronger operation supplies no importance evidence here. Finite/AI-assisted checks are neither mechanical proofs nor independent reproduction.

12 Conclusion

Independent jobs admit exact joint frontiers and a least work curve under optimal protection at every budget. All-program domination supports finite synthesis; specified charges give a complete one-write representation and NP-complete cap decision. Linux supplies a conditional source bound. Native multi-job minimax transfer and general performance benefit remain open.

Data Availability

Code, inputs, complete outcomes/corrections, proofs and replay commands are archived [23], retaining earlier charged/partial-repair results and zero-fee evidence [22]. Licenses accompany code.

References

[1] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Complexity of Computer Computations*, 85–103. https://doi.org/10.1007/978-1-4684-2001-2_9. Reprint: https://www.cs.umd.edu/~gasarch/BLOGPAPERS/Karp.pdf.

[2] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. 2018. Build Systems à la Carte. *PACMPL* 2, ICFP, Article 79, 1–29. https://doi.org/10.1145/3236774. Author version: https://simon.peytonjones.org/assets/pdfs/build-systems-original.pdf.

[3] Ruy Ley-Wild, Umut A. Acar, and Matthew Fluet. 2009. A Cost Semantics for Self-Adjusting Computation. In *POPL*, 186–199. https://doi.org/10.1145/1480881.1480907. Author version: https://ttic.uchicago.edu/~pl/sa-sml/popl09.pdf.

[4] Rachid Guerraoui, Maurice Herlihy, and Bastian Pochon. 2005. Toward a Theory of Transactional Contention Managers. In *PODC*. Author version. https://lpdwww.epfl.ch/upload/documents/publications/neg--509904679f181-pochon.pdf.

[5] Nuno Diegues and Paolo Romano. 2014. Self-Tuning Intel Transactional Synchronization Extensions. In *ICAC*, 209–219. https://www.usenix.org/conference/icac14/technical-sessions/presentation/diegues. Errata.

[6] H. T. Kung and John T. Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 2, 213–226. https://doi.org/10.1145/319566.319567.

[7] Linux contributors. 2024. Linux v6.12, `fs/d_path.c` and `fs/hostfs/hostfs_kern.c`, commit `adc218676eef25575469234709c2d87185ca223a`. Accessed September 11, 2026. https://github.com/torvalds/linux/blob/adc218676eef25575469234709c2d87185ca223a/fs/d_path.c.

[8] Linux contributors. 2024. Linux v6.12, `fs/dcache.c`: name snapshots, `d_exchange`, and tmpfile initialization. Accessed September 11, 2026. https://github.com/torvalds/linux/blob/adc218676eef25575469234709c2d87185ca223a/fs/dcache.c.

[9] Waiman Long. 2013. dcache: Translating dentry into pathname without taking rename_lock. Linux commit `232d2d60aa5469bb097f55728f65146bd49c1d25`. https://github.com/torvalds/linux/commit/232d2d60aa5469bb097f55728f65146bd49c1d25.

[10] .NET Foundation. 2025. *Roslyn source*, commit `6c4a46a31302167b425d5e0a31ea83c9a9aa1d09`. `Workspace.cs`, `Solution.cs`, `SolutionCompilationState.cs` and `SolutionState.cs`. Accessed September 10, 2026. https://github.com/dotnet/roslyn/tree/6c4a46a31302167b425d5e0a31ea83c9a9aa1d09.

[11] .NET Foundation contributors. 2020. Add SetCurrentSolution with solution transformation, pull request 42228. Source review on replayability and event ordering. Accessed September 10, 2026. https://github.com/dotnet/roslyn/pull/42228.

[12] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. https://icml.cc/2012/papers/215.pdf.

[13] Awi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf.

[14] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. https://www.mit.edu/~dbertsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf.

[15] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. https://doi.org/10.1007/978-3-642-22110-1_20. Author publication page and complexity erratum: https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/.

[16] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint* arXiv:1511.07163v1. https://arxiv.org/abs/1511.07163.

[17] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2017. From non-preemptive to preemptive scheduling using synchronization synthesis. *Formal Methods in System Design* 50, 97–139. https://doi.org/10.1007/s10703-016-0256-5.

[18] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html.

[19] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html.

[20] OpenJDK. 2026. ConcurrentHashMap.java, jdk17u source, commit `982d53c227943a511e8d61694feced764b71a6c1`. Accessed September 7, 2026. https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java.

[21] Krzysztof Postek, Ward Romeijnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. https://doi.org/10.1287/opre.2023.0520. Online appendix; author code.

[22] Anonymous Authors. 2026. *Compiling Optimal Retry Policies for Dependent Transformations: Research Artifact*. Version 5 with publication supplement, commit `e0e0110`. https://github.com/research-artifact-archive/research-artifact/tree/e0e011023c0591bbbf422390ad8d29785ad0023f.

[23] Anonymous Authors. 2026. *Retry Synthesis with Bounded Calls and Protected Work: Research Artifact*. Total-work, charged-comparison and native-source supplement, commit `ddf0e22`. https://github.com/research-artifact-archive/research-artifact/tree/ddf0e2261e6a3e091e93e4bb8b12e5a8a4f9f0a4.

[24] Hagit Attiya, Leah Epstein, Hadas Shachnai, and Tami Tamir. 2006. Transactional Contention Management as a Non-Clairvoyant Scheduling Problem. In *PODC*. https://cs.idc.ac.il/~tami/Papers/podc06.pdf.

[25] Dan Alistarh, Syed Kamran Haider, Raphael Kübler, and Giorgi Nadiradze. 2018. The Transactional Conflict Problem. *arXiv preprint* arXiv:1804.00947v1. https://arxiv.org/abs/1804.00947.

[26] Free Software Foundation. 2025. GNU C Library 2.42: x86 lock elision, revision `d2097651`. `elision-lock.c` and `elision-conf.c`. https://github.com/bminor/glibc/tree/d2097651cc57834dbfcaa102ddfacae0d86cfb66/sysdeps/unix/sysv/linux/x86. Official tunables: https://www.gnu.org/software/libc/manual/html_node/Elision-Tunables.html.

[27] OpenJDK. 2025. HotSpot 21.0.8+9 RTM monitor code, revision `54f20959`. `c2_MacroAssembler_x86.cpp` and `globals_x86.hpp`. https://github.com/openjdk/jdk21u/tree/54f2095960f01d957f2335cafa0defb956e13a2c/src/hotspot/cpu/x86.

[28] GNU Compiler Collection. n.d. x86 Transactional Memory Intrinsics. Accessed September 10, 2026. https://gcc.gnu.org/onlinedocs/gcc/x86-transactional-memory-intrinsics.html.

[29] Aditya Dave, Nishanth Venkatesh, and Andreas A. Malikopoulos. 2024. Approximate Information States for Worst-Case Control and Learning in Uncertain Systems. *arXiv preprint* arXiv:2301.05089v2. https://arxiv.org/abs/2301.05089.

[30] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. 2014. Adapton: Composable, Demand-Driven Incremental Computation. In *PLDI*. https://doi.org/10.1145/2594291.2594324. Author version: https://www.cs.tufts.edu/~jfoster/papers/pldi14.pdf.

[31] Mike Ulrich. 2016. Addressing Cascading Failures. In *Site Reliability Engineering*, Chapter 22, Retries. https://sre.google/sre-book/addressing-cascading-failures/.

[32] Envoy contributors. n.d. CircuitBreakers.Thresholds.RetryBudget. Accessed September 10, 2026. https://www.envoyproxy.io/docs/envoy/latest/api-v3/config/cluster/v3/circuit_breaker.proto.html.

[33] Adam Welc, Bratin Saha, and Ali-Reza Adl-Tabatabai. 2008. Irrevocable Transactions and their Applications. In *SPAA*, 285–296. https://doi.org/10.1145/1378533.1378584.

[34] Eugene L. Lawler. 1973. Optimal Sequencing of a Single Machine Subject to Precedence Constraints. *Management Science* 19, 5, 544–546. https://doi.org/10.1287/mnsc.19.5.544. Author technical report ERL-M327 (1971): https://www2.eecs.berkeley.edu/Pubs/TechRpts/1971/Archive/ERL-m-327.pdf.

[35] Christoph Dürr, Thomas Erlebach, Nicole Megow, and Julie Meißner. 2020. An Adversarial Model for Scheduling with Testing. Author preprint arXiv:1709.02592v3. https://arxiv.org/abs/1709.02592v3.

[36] Christoph Damerius, Peter Kling, Minming Li, Chenyang Xu, and Ruilong Zhang. 2023. Scheduling with a Limited Testing Budget: Tight Results for the Offline and Oblivious Settings. Author preprint arXiv:2306.15597v1. https://arxiv.org/abs/2306.15597v1.

[37] Fanny Dufossé, Christoph Dürr, Noël Nadal, Denis Trystram, and Óscar C. Vásquez. 2021. Scheduling with a Processing Time Oracle. Author preprint arXiv:2005.03394v3. https://arxiv.org/abs/2005.03394v3.

[38] Felix Buld and Andreas S. Schulz. 2026. Scheduling with Testing: Competitive Algorithms for Minimizing the Total Weighted Completion Time in the Adversarial Model. Author preprint arXiv:2606.25166v1. https://arxiv.org/abs/2606.25166v1.

[39] Susanne Albers and Alexander Eckl. 2020. Explorable Uncertainty in Scheduling with Non-Uniform Testing Times. Author preprint arXiv:2009.13316. https://arxiv.org/abs/2009.13316.

[40] Susanne Albers and Maximilian Janke. 2021. Online Makespan Minimization with Budgeted Uncertainty. In *WADS*. https://doi.org/10.1007/978-3-030-83508-8_4. Author version: https://maximilian-janke.de/data/BudgetedUncertainty.pdf.

[41] Beng-Hong Lim and Anant Agarwal. 1994. Reactive Synchronization Algorithms for Multiprocessors. In *ASPLOS VI*, 25–35. Author version: https://groups.csail.mit.edu/cag/pub/papers/pdf/reactive.pdf.

[42] Takayuki Usui, Reimer Behrends, Jacob Evans, and Yannis Smaragdakis. 2010. Adaptive Locks: Combining Transactions and Locks for Efficient Concurrency. *Journal of Parallel and Distributed Computing*. https://doi.org/10.1016/j.jpdc.2010.02.006. Author version: https://people.cs.umass.edu/~yannis/adaptive-jpdc.pdf.

[43] Michael Luby, Alistair Sinclair, and David Zuckerman. 1993. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters* 47, 4, 173–180. Author version: https://www.cs.utexas.edu/~diz/pubs/speedup.pdf.